Information Sciences 624 (2023) 200–216

Contents lists available at ScienceDirect

Information Sciences

journal homepage: www.elsevier.com/locate/ins

Self-supervised heterogeneous hypergraph network for knowledge tracing

Tangjie Wu, Qiang Ling *

University of Science and Technology of China, Hefei, Anhui 230027, China

ARTICLE INFO

Article history:
Received 3 May 2022
Received in revised form 24 December 2022
Accepted 27 December 2022
Available online 30 December 2022

Keyword:
Heterogeneous graph neural network
Hypergraph network
Contrastive learning
Self-supervised learning
Knowledge tracing

ABSTRACT

Recently online intelligent education has caught more and more attention, especially due to the global influence of Covid-19. A major task of intelligent education is Knowledge Tracing (KT) which aims to capture students' dynamic status based on their historical interaction records and predict their responses to new questions. However, most existing KT methods suffer from the record data sparsity problem. In reality, there are a huge number of questions in the online database and students can only interact with a very small set of these questions. The records of some questions could be extremely sparse, which may significantly degrade the performance of traditional KT methods. Although recent graph neural network (GNN) based KT methods can fuse graph-structured information and improve the representation of questions to some extent, the pairwise structure of GNN neglects the complex high-order and heterogeneous relations among questions. To resolve the above issues, we develop a novel KT model with the heterogeneous hypergraph network (HHN) and propose an attentive mechanism, including intra- and inter-graph attentions, to aggregate neighbors' information upon HHN. To further enhance the question representation, we supplement the supervised prediction task of KT with an auxiliary self-supervised task, i.e., we additionally generate an augmented view with adaptive data augmentation to implement contrastive learning and exploit the unobserved relations among questions. We conduct extensive experiments on several real-world datasets. Experimental results demonstrate that our proposed method achieves significant performance improvement compared to some state-of-the-art KT methods.

© 2022 Elsevier Inc. All rights reserved.

1. Introduction

With the rapid development of adaptive learning, intelligent tutoring systems (ITS) [1] have been widely implemented. Among research on ITS, more and more attention has been paid to modeling the knowledge state of each student intelligently [2]. As an emerging paradigm of ITS, knowledge tracing (KT) [3] aims to accurately infer the latent state of students based on their historical interactions and predict their responses to a new question. According to the predicted results, teachers can know the mastery of students' knowledge levels in advance and personalize learning plans for each student.

KT methods can date back to the late 1960s. In the early research, the hidden Markov chain was taken to extract the state of students. Bayesian Knowledge Tracing (BKT) [4] is one of typical KT methods. Later, some KT models were built upon Item Response Theory (IRT) [5], like the Difficulty, Ability, and Student History (DASH) model [6]. DASH innovatively takes into

* Corresponding author.
E-mail address: qling@ustc.edu.cn (Q. Ling).

https://doi.org/10.1016/j.ins.2022.12.075
0020-0255/© 2022 Elsevier Inc. All rights reserved.

account the impact of additional factors, such as question difficulty, error rate, response time, on students' final responses. However, the accuracy of these traditional KT methods strongly depends on the statistical estimation of parameters, and may be impracticable nowadays.

Over the past decade, there has emerged a myriad of attempts that implement neural networks to KT. Due to the strong capability of modeling sequential data, neural network-based KT methods generally perform better than traditional ones. For example, Deep Knowledge Tracing (DKT) [7] first introduces recurrent neural network (RNN) [8] to track the state evolution process of students. But this standard sequential architecture of DKT is incapbale of both capturing long-term dependencies and preserving semantic details. To resolve this issue, some KT methods based on memory neural networks, such as Dynamic Key-Value Memory Networks for Knowledge Tracing (DKVMN) [9], were proposed, which supplement an external memory matrix to store the dynamic change of the knowledge state of students.

Later, attention mechanism [10] gradually achieves promising results in NLP field and is implemente to KT. Those attention-based KT models eliminate irrelevant interactions and focus on relevant ones to predict a student's next response. Among them, the Self-Attentive Knowledge Tracing (SAKT) [11] utilizes the attention mechanism to learn the weight of each historical interaction for the target question. Although SAKT shows its effectiveness in capturing early details, it completely ignores the forgetting effects of the memory. Due to the long record cycle in KT, directly applying the attention mechanism may result in significant performance degeneration. In light of this issue, Context-Aware Attentive Knowledge Tracing (AKT) [12] introduces a monotonic attention mechanism that integrates the exponential decay and characterizes the importance of memory decay in modeling long interaction sequences.

Recently, Graph Neural Network (GNN) has emerged as a powerful strategy for analyzing graph-structured data and found many applications in KT. GNN can sufficiently learn node embeddings by aggregating graph attributive and structural features. In the online tutoring systems, KT usually encounters the cold-start or long-tail problem, i.e., most questions are only interacted by a few students. How to tackle such problem and build better representation for questions has attracted much attention. A Graph-based Interaction Model for Knowledge Tracing (GIKT) [13] treats the question-skill data as pair-wise relations and builds the corresponding graph. A joint graph convolutional network based Deep Knowledge Tracing (JKT) [14] models the multi-dimensional relationships of "exercise-to-exercise" and "concept-to-concept" into graph. Pre-training Question Embeddings via Relation Map for Improving Knowledge Tracing (PERM) [15] innovatively exploits two relation maps between questions, including co-occurrence and answer-agreement, and leverages a two-level attention mechanism to fuse multi-faceted information. However, most GNN-based KT methods are susceptible to the subjective tendency of experts within the Q-matrix. To resolve this issue, tracking knowledge proficiency of students with calibrated Q-matrix (CQKT) [16] devises a novel Calibrated Q-matrix-based framework and achieves the state-of-the-art performance. In summary, the graph structure can effectively capture the latent relations among questions and enrich the representation of questions to some extent.

Although GNN-based methods have achieved significant progresses in KT, they still suffer from the following limitations:

(1) The complex heterogeneity is neglected. Heterogeneous information network (HIN) [17] is ubiquitous in the real world. In KT, these recorded questions are usually connected with diverse attributes, like question type, difficulty coefficient, involved skill, which are indispensable to build expressive representations for questions. Obviously, simple homogeneous graphs (question-skill pair) are incapable of characterizing heterogeneous relations.
(2) The high-order relations are neglected. The features encoded just by pair-wise dependencies are not enough to capture higher-order correlations among questions. In reality, multiple questions may cover a common skill, whose relations are no longer dyadic, but higher-order. Traditional GNN can capture k-hop neighbor features by k layer propagation, but is unable to represent the beyond-pairwise semantic structures and patterns.

To address the above limitations, we propose a novel knowledge tracing model and build question embeddings based on the hypergraph [18], which can effectively capture the high-order relations among questions. Those questions, which share a common skill, can be treated as the nodes connected by a hyperedge, and each skill is represented as a hyperedge. Furthermore, we propose a novel intra-graph attention to aggregate neighbor information of the hypergraph. Different from traditional convolution methods [19], which only perform a simple average-pooling over neighbor nodes, our proposed attention can perceive the discrepancy in the impact of nodes and hyperedges and focus on important semantic features. Besides, to quantify the importance of different nodes in the hypergraph, we design a novel projection function, which considers the dynamic deviation under the impact of neighbors.

Although there have been much research on hypergraph learning, little of them has been done on the aforementioned heterogeneity of real scenarios. To resolve this issue, we introduce a heterogeneous information network (HIN) into the hypergraph and learn high-order and heterogeneous relations for question representations simultaneously. It is non-trivial to bridge hypergraph and HIN. The hyperedges of the heterogeneous hypergraph are no longer simplex, but multiple. Each hyperedge denotes a specific relation among questions. To learn the importance of different relations, we propose an inter-graph attention to fuse multiple semantics for representations.

Currently, most KT methods are based on a supervised learning paradigm. However, the performance of those methods may usually be degraded by the data sparsity, i.e., with millions of questions in the database, the observed student-question interactions just provide feedback for a very small set of them, making it very difficult to learn quality representations. To address this limitation, we introduce self-supervised learning (SSL) into our model, which can find supervised signals from

the input data itself and exploit unobserved relations spontaneously to reinforce the question representation. Specifically, we generate multi-views of graphs in different manners to conduct contrastive learning [20,21]. As an effective method of SSL, contrastive learning can learn discriminative embeddings by maximizing the similarity between positive pairs while minimizing the similarity between negative-sampled counterparts. To further enhance the robustness of hypergraph learning, we introduce data augmentation to change the hypergraph structure and generate the augmented view for questions. Different from the previous methods, which drop nodes or edges uniformly, we take the importance of nodes into account and emphasize the important connections. According to the importance of nodes, we adaptively adjust the dropout probability of each node. Thereafter, we unify the prediction task and the self-supervised task, then leverage the multi-task training strategy to jointly optimize the two tasks.

It's worth noting that we just apply a standard recurrent neural network to conduct the prediction task, so as to verify the effectiveness of our proposed framework. Furthermore, our framework only involves upstream representation learning so that it can be well adapted to various downstream prediction models. Our contributions can be summarized as follows.

1. We propose a novel heterogeneous hypergraph network (HHN) for representation learning, which can capture the heterogeneous and high-order relations among questions. Moreover, with the proposed attentive mechanism, including intra- and inter-graph attentions, our HHN can perceive the discrepancy in the impact of nodes and hyperedges, and consequently focus on important connective structures.
2. We introduce a self-supervised task with adaptive augmentation to supplement auxiliary signals for heterogeneous hypergraph modeling, aiming to learn intrinsic patterns and enhance robustness to perturbation.
3. We conduct extensive experiments on several real-world datasets and the results demonstrate our model's superiority compared with some state-of-the-art models.

2. Related work

2.1. Knowledge Tracing

In recent years, deep neural network (DNN) has found more and more applications in diverse fields, including knowledge tracing (KT). KT aims to capture a student's knowledge state according to his interactive questions and predict his response to a new question. DKT [7], the first attempt to employ RNN-based methods in KT, utilizes the LSTM network to model the interactive questions sequences. DKVMN [9] leverages the memory-augmented network to retain long-term dependencies in historical records. Additionally, the attention-based methods, like SAKT [11] and AKT [12], introduce attention mechanisms to highlight these historical interactions that are highly relevant to the target question. ECD [22] proposes an educational context-aware cognitive diagnosis framework to learn the cognitive states from the concerned educational context and students' historical learning records simultaneously. ABKT [23] introduces the ability factor into the learning feedback attribution and analyze the learning process from knowledge and ability. HawkesKT [24] investigates fine-grained temporal cross-effects between different skills in KT and explicitly models these cross-effects by the point process. SEEP [25] fuses questions and concepts to obtain semantic-enhanced question embeddings. GIKT [13] builds a relation graph to extract the underlying relations between questions and skills. PERM [15] exploits two relation maps from two perspectives of co-occurrence and answer-agreement. CQKT [16] proposes a calibrated Q-matrix-based framework to dynamically track knowledge proficiency of students.

Although those recurrent and attentive models have shown their superiority of modeling the sequential data in KT, they all suffer from the inherent data sparsity problem, which makes it insufficient to learn quality representations for a student's historical interactions. Although the GNN-based models can alleviate the sparsity problem to some extent, they may still fail to extract the complex correlations among questions.

2.2. Heterogeneous Graph Neural Network

Recently, the heterogeneous graph has caught much research attention, and shown its great capacity of modeling various types of nodes and relations. The meta-paths in HIN usually represent semantic relations between node pairs. By aggregating features from different meta-paths, the heterogeneous structure can learn more comprehensive presentations for nodes than a homogeneous graph. For example, HAN [26] utilizes hierarchical attention to capture the node-level and semantic-level semantics. Based on HAN, DisenHAN [27] proposes a disentangled heterogeneous graph attention network for recommendation, which considers the entanglement of latent factors stemming from different meta-paths. HetGNN [28] uses an LSTM network to aggregate feature information of sampled neighboring nodes to obtain the ultimate node embeddings. Although the above methods are capable of exploiting multi-faceted semantics, they neglect the beyond-pairwise relations in real scenarios.

2.3. Hypergraph Learning

Benefited from the power of hypergraph in extracting high-order relations, hypergraph learning has achieved great successes in many fields. HGNN [19] is the first method to apply graph convolution to aggregate neighbor features upon the

hypergraph structure. DHGNN [29] develops a dynamic hypergraph network, including dynamic hypergraph construction and hypergraph convolution. DHCF [30] proposes a dual-channel hypergraph with a jump hypergraph convolution (JHConv), which supports the explicit embedding propagation of high-order correlations, to conduct the collaborative filtering task. HyperRec [31] utilizes a sequential hypergraph to generate the dynamic item embeddings for next-item recommendation and DHCN [32] combines hypergraph with a line graph to capture the intra- and inter- information of sessions for session-based recommendation. However, most of them are established in a supervised manner and can not distill supervised signals from data itself.

2.4. Contrastive Learning

As an emerging paradigm of self-supervised learning, contrastive learning aims to learn discriminative representations by contrasting positive and negative pairs. It was firstly used in visual representation learning, in which negative samples are generated by cropping, resizing, rotation, etc. Later, contrastive framework is extended to enhance graph representation learning via self-discrimination. DGI [20] is the first to introduce the InfoMax principle into GNN and augment the original graph by simply shuffling node features. GMI [33] measures the mutual information between a center node and its local patch from node-level and topology-level features. MVGRL [34] applies a graph diffusion network (GDN) to generate additional structural views. GCA [35] optimizes the uniform data augmentation schemes with the adaptive mechanism. HeCo [36] considers two views of an HIN, including network schema and meta-path views, so as to capture local and high-level structures simultaneously. CL4KT [37] introduces contrastive learning into KT and conducts the data augmentation on students' learning history sequence to reveal semantical similarity. But CL4KT cannot fuse the graph structure information and capture the latent relations between nodes.

3. Methodology

In this section, we first formulate the concerned KT task and some definitions related to heterogeneous hypergraph network, and then introduce the proposed methods in detail.

3.1. Problem Formulation

In the KT task, each student's historical record consists of a sequence of questions and responses. Let Q denotes the set of questions. We define the sequence of a student as $\boldsymbol{S_t} = \{(q_1, r_1), (q_2, r_2), \cdots, (q_t, r_t)\}$ in the chronological order. $q_t \in Q$ records the t-th question interacted by the student and $r_t \in \{0, 1\}$ is the corresponding response (1 means correct and 0 means incorrect). Formally, given an observed interactive sequence $\boldsymbol{S_t} = \{(q_1, r_1), (q_2, r_2), \cdots, (q_t, r_t)\}$, our task is to predict the next response r_{t+1} corresponding to a new question q_{t+1}.

3.2. Notations and Definitions

Definition 1 (*Heterogeneous Information Network*). A Heterogeneous Information Network (HIN) [17] is defined as $\mathscr{G} = (\mathscr{V}, \mathscr{E})$, where $\mathscr{V}$ is the node set and $\mathscr{E}$ is the edge set. There are two mapping functions in HIN, including a node type mapping function $\phi : \mathscr{V} \rightarrow \mathscr{A}$ and an edge type mapping function $\psi : \mathscr{E} \rightarrow \mathscr{R}$, where $\mathscr{A}$ and $\mathscr{R}$ represent the node type set and the edge type set, respectively, and satisfy $|\mathscr{A}| + |\mathscr{R}| > 2$.

Definition 2 (*Meta-path*). In the HIN, the meta path Φ is usually defined in the form of $\mathscr{A}_1 \xrightarrow{\mathscr{R}_1} \mathscr{A}_2 \xrightarrow{\mathscr{R}_2} \cdots \xrightarrow{\mathscr{R}_{l-1}} \mathscr{A}_l$. The two node types $\mathscr{A}_1$ and $\mathscr{A}_l$ are connected by the composite relation $\mathscr{R} = \mathscr{R}_1 \circ \mathscr{R}_2 \cdots \mathscr{R}_{l-1}$, where $\circ$ represents the composition operator on relations.

Definition 3 (*Hypergraph*). The hypergraph [18] is denoted as $\mathscr{G}_{hyper} = (\mathscr{V}_h, \mathscr{E}_h)$. $\mathscr{V}_h$ represents the node set and $\mathscr{E}_h$ denotes the hyperedge set. Each hyperedge can connect multiple nodes and the diagonal matrix W represents the weights of the hyperedges. The hypergraph is associated with an incidence matrix H of the size of $|\mathscr{V}_h| \times |\mathscr{E}_h|$, where $H_{v\epsilon} = 1$ if the hyperedge ϵ includes the node v; otherwise $H_{v\epsilon} = 0$. $\mathbf{D}$ and $\mathbf{B}$ are the diagonal degree matrices for nodes and hyperedges, respectively, in which

$$\mathbf{D}_{vv} = \sum_{\epsilon=1}^{|\mathscr{E}_h|} W_{\epsilon\epsilon} H_{v\epsilon} \qquad \mathbf{B}_{\epsilon\epsilon} = \sum_{v=1}^{|\mathscr{V}_h|} H_{v\epsilon}. \tag{1}$$

Based on the above definitions, we integrate the HIN with hypergraph and then implement it to the KT task. As shown in Fig. 1(1), the HIN consists of different nodes types (template (T), question (Q), skill (S)) and relations types (Q → S means the question involves this skill and Q → T means the question belongs to this template). In the heterogeneous hypergraph, each node represents a question, and each hyperedge represents a skill or a template, which connects those questions cov-

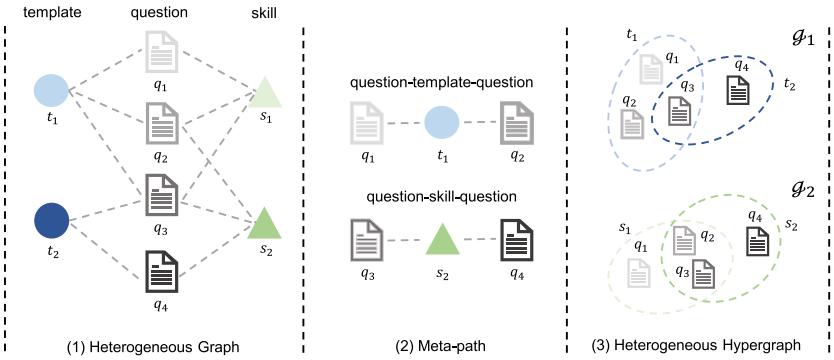


Fig. 1. An illustration of the heterogeneous hypergraph in KT, which includes two types of hyperedges and two subgraphs based on meta-paths Q-T-Q, Q-S-Q. For simplicity, we only show two additional attributes, including skill and template here.

ering the same skill or template. Different hyperedges are connected by their shared nodes. From Fig. 1(3), we can distinctly find that the set-like relations exist among questions. (See Fig. 2)

3.3. Heterogeneous Hypergraph Network

This section introduces the proposed **H**eterogeneous **H**ypergraph **N**etwork (HHN), which aims to learn an informative representation for questions. It is not easy to aggregate neighbors' features over hypergraph. Some existing methods [19] extend the convolution operation for hypergraph, which can be defined as

$$\mathbf{x}_i^l = \sum_{v=1}^{|\mathscr{V}_h|}\sum_{\epsilon=1}^{|\mathscr{E}_h|} H_{i\epsilon} H_{v\epsilon} W_{\epsilon\epsilon} \mathbf{x}_v^{l-1} \mathbf{P}^{l-1}, \quad \mathbf{x}_i^l \in \mathbb{R}^D, \tag{2}$$

where $\mathbf{x}_i^l$ denotes the latent representation of node i in hypergraph, i.e., the embedding of question i, after the l-th layer propagation. At the first layer, $\mathbf{x}_i^0$ is randomly initialized by xavier initialization. $\mathbf{P}^{l-1} \in \mathbb{R}^{D\times D}$ represents the learnable parameter matrix between two convolutional layers and D is the dimension of embeddings. Then the matrix equivalent form of Eq. (2) with row normalization can be represented as

$$\mathbf{X}^l = \mathbf{D}^{-1}\mathbf{H}\mathbf{W}\mathbf{B}^{-1}\mathbf{H}^T\mathbf{X}^l\mathbf{P}^{l-1}, \quad \mathbf{X}^l \in \mathbb{R}^{|\mathscr{V}_h|\times D}, \tag{3}$$

The hypergraph convolution can be viewed as a two-stage information aggregation, where $\mathbf{B}^{-1}\mathbf{H}^T$ and $\mathbf{D}^{-1}\mathbf{H}$ denote the aggregation operation from nodes to hyperedges and the one from hyperedges to nodes, respectively.

It is worth noting that those methods [31,32] usually assign the same weight of hyperedge $W_{\epsilon\epsilon}$ as 1. Obviously, different hyperedges are of different importance to nodes. Just equally treating these hyperedges features may not be reasonable. Besides, the convolution also ignores the discrepancy of nodes in the feature transformation. To further overcome the aforementioned limitations, we propose a novel attentive mechanism to characterize the important connective structures upon HHN.

To explicitly perceive the heterogeneity, we extract the semantic-specific subgraphs $\{\mathscr{G}_1, \mathscr{G}_2, \cdots, \mathscr{G}_K\}$ based on meta-paths from HHN. The discrepant connective structures (node-hyperedge pairs) in subgraphs indicate different latent relations among nodes. Given the initial question embeddings $\mathbf{H}^0 = \left[\mathbf{h}_1^0, \mathbf{h}_2^0, \cdots, \mathbf{h}_{|\mathscr{V}_h|}^0\right] \in \mathbb{R}^{|\mathscr{V}_h|\times D}$, each of which is a trainable embedding associated with the question ID, we apply a graph-specific transformation and project them into different semantic spaces as

$$\mathbf{h}_{i,\mathscr{G}_k}^0 = \mathbf{W}_{\mathscr{G}_k} \cdot \mathbf{h}_i^0, \quad i \in |\mathscr{V}_h|, \tag{4}$$

where $\mathbf{h}_{i,\mathscr{G}_k}^0 \in \mathbb{R}^D$represents the projected feature in the k-th subgraph based on meta-path Φ_k, and $\mathbf{W}_{\mathscr{G}_k} \in \mathbb{R}^{D\times D}$is the projection matrix.

Intra-graph Attention. First, we aggregate the neighbor information in each subgraph to extract single semantic details. Following up the previous methods, we also conduct the aggregation as a two-stage transformation. To reflect the discrepancy in the impact of node-hyperedge pairs, we propose a learnable projection function $f_\epsilon(\cdot) : \mathscr{V}_h \to \mathbb{R}$ to quantify the impact, which measures how important this node will be for a hyperedge. Generally, the weight of the target node for the connected hyperedge is not only affected by the node itself, but also by other neighbor nodes connected in the same hyperedge. Those neighbor nodes usually cause the semantic deviation of the target node to different degrees. We assume

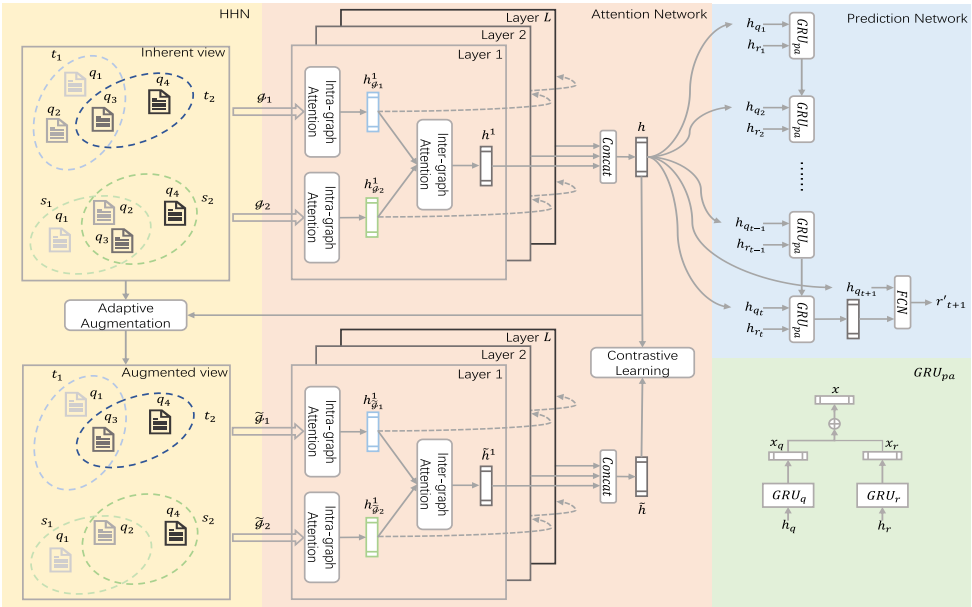


Fig. 2. An overall view of our proposed S^2-HHN. First, based on the inherent view, we learn the representation of question h via intra- and inter-attentions. Then we conduct an adaptive augmentation to generate the augmented view and learn the augmented representation $\tilde{h}$ as h. Afterwards the two representations from different views are utilized to perform the contrastive learning. Finally the representation h is fed into the prediction network to conduct the prediction task.

that if a node is important for the hyperedge, it should keep its neighbors at a semantic "distance", which reflects how well the representation of a target node can be approximated by its neighbors. Those important nodes are usually less affected by their neighbors and can preserve specific semantics. First, we utilize the self-attention mechanism to generate the dynamic representation of node i by considering all neighbor nodes within hyperedge ϵ as

$$\alpha_{ij,\epsilon,\mathscr{G}_k} = \frac{\exp\left(\mathbf{a}^T\left[\mathbf{W}_q\mathbf{h}^{l-1}_{i,\mathscr{G}_k}||\mathbf{W}_k\mathbf{h}^{l-1}_{j,\mathscr{G}_k}\right]\right)}{\sum\limits_{j'\in\mathscr{N}_{\epsilon,\mathscr{G}_k}}\exp\left(\mathbf{a}^T\left[\mathbf{W}_q\mathbf{h}^{l-1}_{i,\mathscr{G}_k}||\mathbf{W}_k\mathbf{h}^{l-1}_{j',\mathscr{G}_k}\right]\right)}, \tag{5}$$

$$\mathbf{d}^{l-1}_{i,\epsilon,\mathscr{G}_k} = \sum_{j\in\mathscr{N}_{\epsilon,\mathscr{G}_k}}\alpha_{ij,\epsilon,\mathscr{G}_k}\cdot\mathbf{h}^{l-1}_{j,\mathscr{G}_k}, \tag{6}$$

where $||$ stands for the concatenation operation, $\mathbf{W}_q, \mathbf{W}_k \in \mathbb{R}^{D\times D}$ are transformation matrices and $\mathbf{a}^T \in \mathbb{R}^{2D}$ is the attention vector, which transforms the concatenated embeddings into a scalar. $\mathbf{h}^{l-1}_{i,\mathscr{G}_k}$ denotes the refined embedding of question node i after the $(l-1)$-th layer, and $\mathscr{N}_{\epsilon,\mathscr{G}_k}$ represents the set of nodes that are connected by hyperedge ϵ in subgraph $\mathscr{G}_k$. Note that we do not introduce any nonlinear transformation in the attention, which would lead to worse performance in our results. Then the projection function is defined as

$$f_\epsilon\left(\mathbf{h}^{l-1}_{i,\mathscr{G}_k}\right) = \frac{\frac{1}{|\mathscr{N}_{\epsilon,\mathscr{G}_k}|-1}\sum\limits_{j\in\mathscr{N}_{\epsilon,\mathscr{G}_k}\setminus i}\|\mathbf{d}^{l-1}_{i,\epsilon,\mathscr{G}_k}-\mathbf{h}^{l-1}_{j,\mathscr{G}_k}\|_2}{1+\|\mathbf{d}^{l-1}_{i,\epsilon,\mathscr{G}_k}-\mathbf{h}^{l-1}_{i,\mathscr{G}_k}\|_2}, \tag{7}$$

where $\|\cdot\|_2$ measures the Euclidean distance between two vectors. When the numerator of Eq. (7) is large, the target node keeps distinct semantics from its neighbors and is hardly approximated, which indicates its indispensability for the hyperedge. Moreover, when the denominator of Eq. (7) is small, the semantics deviation between $\mathbf{d}_{i,\epsilon,\mathscr{G}_k}$ and $\mathbf{h}_{i,\mathscr{G}_k}$ is not prominent and the impact of neighbors is weak for the target node. Next, we normalize those weight coefficients calculated by the projection function via softmax and aggregate the node features with them to generate the representation of hyperedge ϵ as

$$\beta^l_{i\epsilon,\mathscr{G}_k} = \frac{\exp\left(f_\epsilon\left(\mathbf{h}^{l-1}_{i,\mathscr{G}_k}\right)/\eta\right)}{\sum\limits_{i'\in\mathscr{N}_{\epsilon,\mathscr{G}_k}}\exp\left(f_\epsilon\left(\mathbf{h}^{l-1}_{i',\mathscr{G}_k}\right)/\eta\right)}, \tag{8}$$

$$\mathbf{z}^l_{\epsilon,\mathscr{G}_k} = \sum_{i\in\mathscr{N}_{\epsilon,\mathscr{G}_k}}\beta^l_{i\epsilon,\mathscr{G}_k}\cdot\mathbf{h}^{l-1}_{i,\mathscr{G}_k}, \tag{9}$$

where η is a temperature parameter. The above operation implements the information aggregation from nodes to hyperedges, and focuses on the important nodes for hyperedge based on the hypergraph structure.

After that, we utilize the attention mechanism to capture the relevance among the hyperedges, which are connected with the target node, and transform the information from hyperedges to nodes. For node i, the importance of hyperedge ϵ's features is calculated as

$$\chi^l_{\epsilon i,\mathscr{G}_k} = \frac{\exp\left(\varphi\left(\mathbf{a}^T_{\mathscr{G}_k}\left[\mathbf{W}_v\mathbf{h}^{l-1}_{i,\mathscr{G}_k}||\mathbf{W}_e\mathbf{z}^l_{\epsilon,\mathscr{G}_k}\right]\right)\right)}{\sum_{\epsilon'\in\mathscr{E}_{i,\mathscr{G}_k}}\exp\left(\varphi\left(\mathbf{a}^T_{\mathscr{G}_k}\left[\mathbf{W}_v\mathbf{h}^{l-1}_{i,\mathscr{G}_k}||\mathbf{W}_e\mathbf{z}^l_{\epsilon',\mathscr{G}_k}\right]\right)\right)}, \tag{10}$$

where $\mathbf{a}^T_{\mathscr{G}_k} \in \mathbb{R}^{2D}$ is the attention vector, and $\mathbf{W}_v, \mathbf{W}_e \in \mathbb{R}^{D\times D}$ are projection matrices, which project the representations of nodes and hyperedges into the same feature space. These parameters are shared in each layer to avoid overfitting. $\mathscr{E}_{i,\mathscr{G}_k}$ denotes the set of hyperedges that contain node i in subgraph $\mathscr{G}_k$. φ is the LeakyReLU activation function. Similarly we can obtain the semantic representations for nodes by aggregating the corresponding hyperedges as

$$\mathbf{h}^l_{i,\mathscr{G}_k} = \sum_{\epsilon\in\mathscr{E}_{i,\mathscr{G}_k}} \chi^l_{\epsilon i,\mathscr{G}_k}\cdot\mathbf{z}^l_{\epsilon,\mathscr{G}_k}. \tag{11}$$

Inter-graph Attention. After independently aggregating the information within different subgraphs, we can obtain the heterogeneous semantic representations $\left\{\mathbf{h}^l_{i,\mathscr{G}_1}, \mathbf{h}^l_{i,\mathscr{G}_2}, \cdots, \mathbf{h}^l_{i,\mathscr{G}_K}\right\}$ for node i, which reflect node features from diverse aspects. To learn the importance of relations in different subgraphs, we conduct an inter-graph attention to fuse those semantic-specific embeddings at each layer. As the hypergraph can be represented identically by the contained hyperedges, we utilize a graph pooling function, which averages all hyperedges embeddings, to generate the hypergraph representation. Generally, this representation can be explained as the importance of each hypergraph. Based on this setup, we can measure the weight of subgraph $\mathscr{G}_k$ for node i as

$$w_{i,\mathscr{G}_k} = \left(\mathbf{W}\cdot\mathbf{h}^l_{i,\mathscr{G}_k} + \mathbf{b}\right)^T \cdot \left\{\frac{1}{|\mathscr{E}_{\mathscr{G}_k}|}\sum_{\epsilon\in\mathscr{E}_{\mathscr{G}_k}} \tanh\left(\mathbf{W}_{\mathscr{G}_k}\cdot\mathbf{z}^l_{\epsilon,\mathscr{G}_k} + \mathbf{b}_{\mathscr{G}_k}\right)\right\}, \tag{12}$$

$$\gamma_{i,\mathscr{G}_k} = \frac{\exp\left(w_{i,\mathscr{G}_k}\right)}{\sum_{k'=1}^{K}\exp\left(w_{i,\mathscr{G}_{k'}}\right)}, \tag{13}$$

where $\mathbf{W}_{\mathscr{G}_k}, \mathbf{W} \in \mathbb{R}^{D\times D}$ are the weight matrices and $\mathbf{b}_{\mathscr{G}_k}, \mathbf{b} \in \mathbb{R}^D$ are the bias vectors. Note that $\mathbf{W}_{\mathscr{G}_k}, \mathbf{b}_{\mathscr{G}_k}$ are graph-specific parameters, enabling that distinct semantics for different subgraphs can be captured. With the learned coefficients γ, we can form the hybrid representation as

$$\mathbf{h}^l_i = \sum_{k=1}^{K}\gamma_{i,\mathscr{G}_k}\cdot\mathbf{h}^l_{i,\mathscr{G}_k}. \tag{14}$$

After the information aggregation of L layers, we concatenate those embeddings obtained at each layer to generate the final representation of node i as

$$\mathbf{h}_i = \mathbf{W}_L\cdot\left[||_{l=1}^{L}\mathbf{h}^l_i\right], \tag{15}$$

where $\mathbf{W}_L \in \mathbb{R}^{D\times LD}$ is the layer weight matrix.

3.4. Downstream Prediction Network

In this component, we adopt the standard RNN with Gated Recurrent Units (GRU) [38] to extract a student's knowledge state from his historical sequence. The dependencies between interactive questions and the dynamics of the student's responses are independently tracked by a parallel GRU as

$$\mathbf{x}_{q_t} = \mathrm{GRU}_q\left(\mathbf{h}_{q_1}, \mathbf{h}_{q_2}, \cdots, \mathbf{h}_{q_t}\right), \mathbf{x}_{q_t} \in \mathbb{R}^{D_r}, \tag{16}$$

$$\mathbf{x}_{r_t} = \mathrm{GRU}_r\left(\mathbf{h}_{r_1}, \mathbf{h}_{r_2}, \cdots, \mathbf{h}_{r_t}\right), \mathbf{x}_{r_t} \in \mathbb{R}^{D_r}, \tag{17}$$

where $\mathbb{R}^{D_r}$ denotes the size of the hidden layer of GRU. $\mathbf{h}_{q_t}$ is the generated embedding of interactive question q_t and $\mathbf{h}_{r_t}$ is the corresponding response embedding to indicate correct or incorrect. The t-th step knowledge state $\mathbf{x}_t$ is produced by the summation of two hidden states as

$$\mathbf{x}_t = \mathbf{x}_{q_t} + \mathbf{x}_{r_t}. \tag{18}$$

While predicting a student's response, we take the target question embedding into consideration. Next, we utilize a fully-connected network (FCN) to generate the predicted probability $r'_{t+1} \in [0,1]$ according to which the student can correctly answer the target question. For training these learnable parameters, we adopt the following binary cross-entropy loss as the objective function

$$r'_{t+1} = \sigma\big(\mathrm{FCN}\big([\mathbf{x}_t || \mathbf{h}_{q_{t+1}}]\big)\big), \tag{19}$$

$$\mathscr{L}_{main} = -\sum\big(r_{t+1}\log(r'_{t+1}) + (1-r_{t+1})\log(1-r'_{t+1})\big), \tag{20}$$

where σ is the sigmoid activation function and r_{t+1} stands for the student's true response at step $t+1$. In this paper, we choose $\mathscr{L}_{main}$ as the main supervised task.

3.5. Adaptive Data Augmentation on HHN

Although contrastive learning has been widely applied in CV and NLP, it has rarely been explored in the graph field. It is still quite challenging to generate proper representation views for contrastive learning. Some existing methods construct multiple views based on multi-hop neighbors from the perspective of inherent structure. While those models usually tend to favor those observed connections, they may ignore the intrinsic (unobserved) relations underneath the input graph. The impact of observed structures is magnified in the layered information aggregation, which leads to the degree biases in learning representations and the vulnerability in facing noisy perturbations. To alleviate the above problem, we introduce a novel data augmentation paradigm to generate the augmented view for HHN. Different from most of the augment-based methods, which just remove nodes or edges equivalently and change the graph structure in a uniform manner, we propose an adaptive scheme to adjust the dropout probability of each node according to its importance.

Graph-specific Topological Augmentation. In view of the heterogeneous relations in different subgraphs, we propose a graph-specific manner to conduct topological augmentation. For each subgraph of HHN, we independently discard each node from the connected hyperedge with the probability ρ. Specifically, the probability $\rho_{i\epsilon,\mathscr{G}_k}$ of removing node i from hyperedge ϵ is calculated according to the node importance in Eq. (5–9) as

$$\rho_{i\epsilon,\mathscr{G}_k} = \frac{\overline{\rho_{\epsilon,\mathscr{G}_k}} - \beta_{i\epsilon,\mathscr{G}_k}}{\overline{\rho_{\epsilon,\mathscr{G}_k}} - \underline{\rho_{\epsilon,\mathscr{G}_k}}} \cdot \rho_\tau + \rho_\lambda, \tag{21}$$

$$\overline{\rho_{\epsilon,\mathscr{G}_k}} = \mathrm{Max}\Big\{\beta_{i\epsilon,\mathscr{G}_k} | i \in \mathscr{N}_{\epsilon,\mathscr{G}_k}\Big\}, \qquad \underline{\rho_{\epsilon,\mathscr{G}_k}} = \mathrm{Min}\Big\{\beta_{i\epsilon,\mathscr{G}_k} | i \in \mathscr{N}_{\epsilon,\mathscr{G}_k}\Big\}, \tag{22}$$

where ρ_τ, ρ_λ are the hyperparameters that control the range of dropout probability. Note that we adopt the final representation $\mathbf{h}_i$ to calculate the node weight $\beta_{i\epsilon,\mathscr{G}_k}$.

For those nodes with lower importance, we expect to assign a higher dropout probability, so as to corrupt the unimportant connections and make the representation insensitive to structure perturbation. While for those nodes with higher importance, the lower dropout probability can better preserve intrinsic relations and capture underlying semantics. It is worth noting that the probability of removing the node contained in different hyperedges is calculated independently, and can be adjusted adaptively according to the features of neighbors within the hyperedge.

Given the original subgraphs $\{\mathscr{G}_1, \mathscr{G}_2, \cdots, \mathscr{G}_K\}$, which are deemed as inherent views, we apply the graph-specific topological augmentation to corrupt the node-hyperedge pairs in those hypergraphs randomly, and generate the augmented views of subgraphs as $\{\tilde{\mathscr{G}}_1, \tilde{\mathscr{G}}_2, \cdots, \tilde{\mathscr{G}}_K\}$. With the above attention mechanism, we update the representation of initial question embeddings by aggregating the neighbors in inherent and augmented views, respectively,

$$\mathbf{H}^{inh} = \mathrm{HHN}\Big(\mathbf{H}^0, \{\mathscr{G}_1, \mathscr{G}_2, \cdots, \mathscr{G}_K\}\Big), \tag{23}$$

$$\mathbf{H}^{aug} = \mathrm{HHN}\Big(\mathbf{H}^0, \{\tilde{\mathscr{G}}_1, \tilde{\mathscr{G}}_2, \cdots, \tilde{\mathscr{G}}_K\}\Big). \tag{24}$$

The parameters are shared in the two independent processes. Note that we apply the augmentation on graph structure per batch in one epoch during training the model in order to supplement diverse neighbors for contrastive learning.

3.6. Contrastive Learning

After obtaining the refined question embeddings $\mathbf{H}^{inh}$ and $\mathbf{H}^{aug}$ based on the inherent and augmented views, we integrate contrastive learning into HHN to distinguish embeddings from different views. Following [20,39], the views of the same nodes are regarded as positive pairs and the views of any different nodes as negative pairs. However, the above way of generating negative pairs might lead to suboptimal results. In HHN, nodes are contained by multiple types of hyperedges, which indicate various relations among them. The correlations among these nodes usually lead to the similarity of their representations to some extent, thus causing the semantic blending for downstream tasks. For those nodes that are not included in any hyperedge with the target node, they are often unrelated to the target node and easily distinguishable from the positive pairs. Moreover, conducting negative pairs with them is not conducive to identifying the subtle discrepancies. To avoid this difficulty, we just consider those nodes being contained in one or more hyperedges with the target node as negative samples,

which enforce the model to discriminate the positive pairs from samples with high similarity. So the negative sample set of node i can be defined as

$$\mathcal{N}_i^{inh} = \{j|(i,j) \in \mathcal{E}_{\mathcal{G}_k}, \mathcal{G}_k \in \{\mathcal{G}_1, \mathcal{G}_2, \cdots, \mathcal{G}_K\}\}, \tag{25}$$

$$\mathcal{N}_i^{aug} = \left\{j|(i,j) \in \mathcal{E}_{\tilde{\mathcal{G}}_k}, \tilde{\mathcal{G}}_k \in \{\tilde{\mathcal{G}}_1, \tilde{\mathcal{G}}_2, \cdots, \tilde{\mathcal{G}}_K\}\right\}, \tag{26}$$

where $\mathcal{E}_{\mathcal{G}_k}, \mathcal{E}_{\tilde{\mathcal{G}}_k}$ are the sets of hyperedges in the inherent and augmented views, respectively. $(i,j) \in \mathcal{E}_{\mathcal{G}_k}$ means that nodes i and j are included in one hyperedge simultaneously. Formally, we utilize the InfoNCE objective as the contrastive loss to contrast node representations from different views. The objective for node i is defined as

$$\mathcal{L}_i^{inh} = -\log \frac{\exp\left(s\left(\mathbf{h}_i, \tilde{\mathbf{h}}_i\right)/r\right)}{\exp\left(s\left(\mathbf{h}_i, \tilde{\mathbf{h}}_i\right)/r\right) + \sum_{j \in \mathcal{N}_i^{inh}} \exp\left(s\left(\mathbf{h}_i, \mathbf{h}_j\right)/r\right) + \sum_{j \in \mathcal{N}_i^{aug}} \exp\left(s\left(\mathbf{h}_i, \tilde{\mathbf{h}}_j\right)/r\right)}, \tag{27}$$

where $s(\cdot)$ is the cosine similarity function to measure the similarity between two vectors and r denotes a temperature parameter, and $\mathbf{h}_i, \tilde{\mathbf{h}}_i$ represent the embeddings of question i in embeddings matrices $\mathbf{H}^{inh}$ and $\mathbf{H}^{aug}$, respectively. The fusion of inter- and intra-views contributes to the learning of cross-structural features.

According to the symmetry in the above two views, we can define the contrastive loss $\mathcal{L}_i^{aug}$ similarly, where the target embedding is from the augmented view. The overall objective is given as

$$\mathcal{L}_{ssl} = \frac{1}{|\mathcal{V}_h|} \sum_{i \in \mathcal{V}_h} \left(\mathcal{L}_i^{inh} + \mathcal{L}_i^{aug}\right). \tag{28}$$

Finally, we unify the main prediction task and the self-supervised task, and optimize the two tasks with a multi-task training strategy. The joint objective is defined as

$$\mathcal{L} = \mathcal{L}_{main} + \lambda \mathcal{L}_{ssl}, \tag{29}$$

where λ is the hyperparameter that controls the contribution of the self-supervised task.

4. Experiments

In this section, we take several public datasets to evaluate the performance of our proposed model and some state-of-the-art models. Particularly, two versions of the proposed HHN are investigated in the experiments, where HHN represents the vanilla version (without self-supervised learning) and S^2-HHN represents the self-supervised version. Furthermore, we conduct some ablation studies to verify the effectiveness of different components of HHN.

4.1. Datasets

We evaluate different models on several real-world knowledge tracing datasets, including ASSISTments2009[1], ASSISTments2017[2], EdNet[3] and JunyiAcademy[4], which are widely used in the educational data mining field to predict students' performance. The statistics of the datasets are summarized in Table 1.

ASSISTments2009. This dataset was collected from ASSISTments online education platform during the school year 2009–2010, which has two versions: Skill-builder and Non-Skill Builder. we select the updated Skill-builder version to conduct our experiments and remove duplicated records. Besides, we remove those students who have less than 3 historical records.

ASSISTments2017. This dataset was collected from the same tutoring platform as ASSISTments2009 and was used in Data Mining Competition 2017. It includes 11-year data spanning from middle schools for mathematics. We remove the data recorded as '*noskill*' in the skill field.

EdNet. This dataset was collected from a multi-platform AI tutoring over 2 years with more than 780 K users in Korea, which contains various features of students' actions, such as the learning materials, the response, the time spent on solving a given question or reading through experts' commentary. Note that EdNet is composed of a total of 131441538 interactions collected from 784,309 students, which is too large for training. So we randomly select 5000 students and their interactive records to conduct experiments.

JunyiAcademy. The dataset contains the problem log and exercise-related information on the Junyi Academy. Besides, the platform collects the annotations of exercise relationships for building models. Similarly, considering that the dataset is too large, which contains 25925992 interactions collected from 247606 users, we randomly select 5000 users and their interactive records.

[1] https://sites.google.com/site/assistmentsdata/home/assistment-2009–2010-data
[2] https://sites.google.com/view/assistmentsdatamining/dataset
[3] https://github.com/riiid/ednet
[4] https://pslcdatashop.web.cmu.edu/Files?datasetId = 1198

Table 1
Dataset statistics.

Dataset	Question	Attribute	Student	Record	Realtion	Meta-path
ASSISTments2009	17751	Skill:123 Template:658	4151	325637	Q-S: 21246 Q-T: 17751	Q-S-Q Q-T-Q
ASSISTments2017	2210	Skill:101 Assign:236	1709	942816	Q-S: 2803 Q-A: 16820	Q-S-Q Q-A-Q
EdNet	12120	Skill:189 Bundle: 8853	5000	6212117	Q-S: 28758 Q-B: 13169	Q-S-Q Q-B-Q
JunyiAcademy	509	Topic:39 Area: 9	5000	346918	Q-T: 509 Q-A: 509	Q-T-Q Q-A-Q

The following attributes of questions are considered.

- Skill (S). The cognitive skill associated with the question. A question may cover multiple skills.
- Template (T). The template of the ASSISTments platform. Similar questions are collected in the same template.
- Assign (A). A unique tag is used for identifying an assignment with questions in ASSISTments2017.
- Bundle (B). The expert-annotated bundle for each question. Each bundle can contain from 1 to 5 questions.
- Topic (T). The topic of each question represents a predominant concept in the knowledge map.
- Area (A). The knowledge area that question covers. Each area may contain several topics.

4.2. Baselines

To verify the effectiveness of our method, we implement some state-of-the-art methods as baselines for evaluation.

DKT [7] is the first method to introduce the deep neural network into the KT field, which uses the long short term memory (LSTM) network to model the evolution of student state.

DKT+ [40] is an extension of traditional DKT method, which considers students' historical interaction sequence and forgetting behavior comprehensively.

DKVMN [9] utilizes the dynamic memory network and stores a student's mastery of each skill by an external memory matrix instead of a single hidden vector. The key matrix and value matrix are updated by read and write operations according to each student's historical sequences, which make the representation of his latent state more interpretable.

SAKT [11] introduces a self-attention mechanism to capture the relevance among interactive questions and focus on those historical interactions that are highly relevant to the target question.

AKT [12] uses two-layer monotonic attention with Rasch embeddings, which integrates the exponential decay and a context-aware relative distance measure, to model sequential interaction.

GIKT [13] adopts a graph structure to enrich the representation of questions and utilizes the GCN to capture question-skill relations. Then, it applies an LSTM network to update the student's latent knowledge state.

HawkesKT [24] takes the point process to explicitly model the temporal cross-skill impact and includes two components, including mutual excitation and kernel function, to measure the degree of cross-effects.

CL4KT [37] applies the data augmentation method to generate two correlated views of a student's history sequences and introduces contrastive learning to learn discriminative representations for history sequences.

4.3. Experimental Settings

In our experiments, we set the input sequence length as 200. If a student has more than 200 records, we truncate the whole sequence into several sub-sequences. We fix the size of the hidden layer of RNN as $D_r = 320$. Considering that the variation of some parameters has great impact on results, we conduct the ablation study to search their optimal values. The dimension of question embeddings is searched from $\{64, 128, 256, 320, 512\}$, the hyperparameters ρ_τ, ρ_λ are searched from the range of $\{0, 0.1, 0.2, 0.3, 0.4, 0.5\}$ and λ from $\{0.01, 0.05, 0.1, 0.5, 1, 2, 5\}$. The temperature parameters η and r are tuned within the range of $\{0.1, 0.2, 0.5, 1.0, 2.0\}$. Each dataset is divided based on students, among which 80% of students' records are used for training and validating, and the rest 20% for testing. The learnable parameters in the network are set with Xavier parameter initialization and are optimized by Adam algorithm. The learning rate is tuned among $\{0.0001, 0.0005, 0.001, 0.005, 0.01, 0.05, 0.1\}$ and the batch size is tuned among $\{32, 64, 128, 256, 512\}$. All methods are implemented in Tensorflow1.14 and follow the same experimental settings for fair comparison.

4.4. Results

To follow up the previous work, we also evaluate the performance by Area Under Curve (AUC) and Accuracy (ACC) for classification and Root Mean Square Error (RMSE) for regression. We compare our model with different baselines across all datasets. The experimental results of overall performance are reported in Table 2, where we highlight the best results

in boldface and the second best results in underline. As shown in Table 2, our S^2-HHN model improves the AUC score by 5.54%, 4.41%, 3.06%, 4.53% over DKT, 3.82%, 2.92%, 1.99%, 2.22% over GIKT, 1.92%, 1.44%, 0.91%, 2.53% over AKT, 2.15%, 1.58%, 0.31%, 0.99% over HawkesKT and 1.43%, 1.47%, 0.59%, 1.32% over CL4KT; the ACC score by 6.17%, 6.36%, 3.67%, 3.55% over DKT, 4.76%, 4.61%, 2.08%, 1.86% over GIKT, 2.49%, 1.58%, 0.98%, 1.28% over AKT, 2.34%, 1.94%, 0.44%, 0.61% over HawkesKT and 1.80%, 1.66%, 0.75%, 0.40% over CL4KT (60% training). Even without self-supervised learning, our proposed HHN still achieves competitive performance with other baselines. In particular, the performance improvement on ASSISTments2009 is significant. By taking a deep look at this dataset, we find that its records are sparser than those of other datasets, which directly leads to inadequate representation learning and limits the performance of some baselines. Owing to the great capacity of HHN in capturing high-order and heterogeneous relations, we can still learn an expressive representation for questions from extremely sparse data. As the proportion of the training set decreases, the improvement of our model becomes more significant, which also confirms the efficiency of our model in dealing with the sparse data problem.

In Ednet, which has the most records among all datasets, the standard DKT almost achieves performance comparable to other more complex models. This could convince us that the downstream prediction network may not be the primary reason to lead to such suboptimal results.

In ASSISTments2009, whose records are the most sparse among all datasets, the self-supervised methods, like CL4KT and S^2-HHN, outperform other baselines, which is in line with our analysis that the sparsity of historical data might hinder the student's knowledge state modeling, and introducing self-supervision signals could address it. However, CL4KT only focuses on learning the representation of historical sequences by contrasting different views, omitting the heterogeneous and high-order relations among questions, which would fail to build sufficient representation for each question. Our proposed S^2-HHN conducts the graph contrastive learning to exploit unobserved relations in HHN and greatly promotes the performance of the downstream prediction task.

Compared with traditional graph-based methods, our model has two advantages: (1) it can capture different semantic details from heterogeneous information and learn more informative representations for questions. By modeling various relations in HHN, the connections between distant questions can be well exploited. (2) it utilizes hypergraph topology to extract the beyond-pairwise relations among questions. In real scenarios, similar questions are usually contained in one set. Traditional graph structure usually only supports pairwise relations between questions and is incapable of revealing such set-like relations. The hypergraph structure, in which each hyperedge can connect multiple nodes, can naturally encode high-order correlations.

Meanwhile, self-supervised learning has great potential to further improve HHN. Compared with the vanilla version, the self-supervised version shows decent performance improvement in all the cases. Particularly, when dealing with those sparse records, self-supervised learning achieves significant gains. If we reduce the percentage of training set to 20%, the improvement of S^2-HHN in these datasets increases from 1.77%, 1.04%, 0.29%, 1.10% to 2.45%, 3.29%, 1.18%, 1.28% compared with HHN in term of the AUC score, which also demonstrates that the sparser are the records, the more necessary is the self-supervised learning.

4.5. Ablation Study

To verify the effectiveness of components of the proposed S^2-HHN, we conduct some ablation studies to evaluate their contributions in term of the AUC score. As the superiority of HHN results from the joint effects of the heterogeneous network and the hypergraph structure, we develop several variants of HHN to validate their effectiveness for the final performance.

- GCN [41]: adopts a graph convolutional network to aggregate neighbors' features. Considering that it is designed for homogeneous graphs, we test all the meta-paths for GCN and select the optimal performance for comparison.
- GAT [42]: adopts the attention mechanism to aggregate neighbors' features on the homogeneous graph. Similarly, we test all the meta-paths for GAT and show the optimal performance.
- HyperGCN: applies a hypergraph convolutional network [19] on the homogeneous hypergraph. We test all types of hyperedges based on meta-paths and show the optimal performance.
- HeteroGCN: applies a graph convolutional network [41] on the heterogeneous graph. Here we aggregate neighbors' features on each meta-path and average them to generate the final representation.
- HAN [26]: employs node-level and semantic-level attentions simultaneously on the heterogeneous graph, and assigns different importance to each node and meta-path.
- HHN-S: employs the proposed intra-graph attention on the homogeneous hypergraph. We test all types of hyperedges and show the optimal performance.

As shown in Table 3, we see that HHN outperforms other variants.

1. Compared with the convolution methods (GCN, HyperGCN, HeteroGCN), which simply average over neighbors, attention can adaptively adjust the weights between nodes and hyperedges and focus on important connective structures. By assigning different importance to nodes and hyperedges, the proposed HHN can learn expressive representations for questions.

Table 2
Performance of all methods on the four datasets. The best results are in bold face and the second best results are underlined (60% training).

Dataset	ratio	ASSISTments 2009			ASSISTments 2017			Ednet			JunyiAcademy		
Metrics		AUC	ACC	RMSE	AUC	ACC	RMSE	AUC	ACC	RMSE	AUC	ACC	RMSE
DKT	20%	0.7738	0.7321	0.4470	0.6904	0.6433	0.4884	0.7088	0.6825	0.4836	0.7105	0.6819	0.4747
	40%	0.7920	0.7598	0.4315	0.7238	0.6687	0.4679	0.7229	0.7012	0.4678	0.7227	0.6994	0.4683
	60%	0.8012	0.7832	0.4248	0.7473	0.7096	0.4585	0.7445	0.7288	0.4452	0.7443	0.7203	0.4479
DKT+	20%	0.7756	0.7328	0.4397	0.6939	0.6449	0.4881	0.7130	0.6901	0.4829	0.7152	0.6833	0.4740
	40%	0.7963	0.7635	0.4285	0.7171	0.6592	0.4732	0.7287	0.7035	0.4635	0.7196	0.6950	0.4702
	60%	0.8099	0.7856	0.4231	0.7388	0.7017	0.4641	0.7493	0.7270	0.4463	0.7381	0.7188	0.4526
DKVMN	20%	0.7780	0.7481	0.4362	0.6753	0.6528	0.4925	0.6983	0.6756	0.4910	0.7083	0.6720	0.4811
	40%	0.7924	0.7708	0.4320	0.7080	0.6797	0.4755	0.7198	0.7003	0.4711	0.7170	0.6875	0.4732
	60%	0.8114	0.7901	0.4212	0.7406	0.7215	0.4630	0.7492	0.7290	0.4449	0.7349	0.7126	0.4555
GIKT	20%	0.7962	0.7665	0.4249	0.7189	0.6701	0.4718	0.7356	0.7184	0.4603	0.7300	0.7120	0.4561
	40%	0.8089	0.7863	0.4207	0.7395	0.7052	0.4637	0.7490	0.7308	0.4432	0.7456	0.7261	0.4427
	60%	0.8145	0.7937	0.4190	0.7581	0.7331	0.4563	0.7523	0.7402	0.4410	0.7611	0.7323	0.4365
SAKT	20%	0.7402	0.7026	0.4692	0.6204	0.5942	0.5091	0.6534	0.6322	0.4971	0.6371	0.6202	0.4952
	40%	0.7701	0.7413	0.4513	0.6531	0.6178	0.4968	0.6678	0.6498	0.4954	0.6833	0.6507	0.4836
	60%	0.7932	0.7521	0.4358	0.7009	0.6790	0.4870	0.6989	0.6830	0.4865	0.7102	0.6744	0.4771
AKT	20%	0.7830	0.7611	0.4421	0.7049	0.6800	0.4856	0.7164	0.6973	0.4807	0.7157	0.6851	0.4744
	40%	0.8071	0.7892	0.4228	0.7438	0.7205	0.4621	0.7382	0.7229	0.4570	0.7402	0.7234	0.4495
	60%	0.8297	0.8113	0.4039	0.7692	0.7430	0.4538	0.7604	0.7482	0.4354	0.7588	0.7365	0.4390
HawkesKT	20%	0.7852	0.7619	0.4417	0.7188	0.6712	0.4715	0.7153	0.6879	0.4855	0.7325	0.7143	0.4537
	40%	0.8123	0.7933	0.4203	0.7502	0.7271	0.4606	0.7405	0.7208	0.4598	0.7608	0.7370	0.4382
	60%	0.8278	0.8125	0.4038	0.7682	0.7404	0.4557	0.7649	0.7523	0.4340	0.7703	0.7414	0.4305
CL4KT	20%	0.7948	0.7661	0.4257	0.7210	0.6728	0.4705	0.7389	0.7236	0.4552	0.7502	0.7291	0.4400
	40%	0.8151	0.7975	0.4180	0.7455	0.7194	0.4617	0.7517	0.7405	0.4379	0.7617	0.7399	0.4326
	60%	0.8337	0.8168	0.4013	0.7690	0.7425	0.4544	0.7628	0.7500	0.4318	0.7679	0.7430	0.4291
HHN	20%	0.8079	0.7856	0.4220	0.7238	0.6811	0.4673	0.7403	0.7225	0.4563	0.7511	0.7304	0.4397
	40%	0.8276	0.8109	0.4097	0.7498	0.7270	0.4592	0.7526	0.7402	0.4370	0.7631	0.7389	0.4339
	60%	0.8309	0.8143	0.4035	0.7722	0.7499	0.4516	0.7651	0.7519	0.4306	0.7696	0.7425	0.4311
S^2-HHN	20%	0.8277	0.8120	0.4062	0.7476	0.7258	0.4608	0.7491	0.7278	0.4515	0.7607	0.7370	0.4358
	40%	0.8341	0.8206	0.3966	0.7592	0.7401	0.4544	0.7600	0.7493	0.4327	0.7714	0.7446	0.4303
	60%	**0.8456**	**0.8315**	**0.3883**	**0.7803**	**0.7548**	**0.4330**	**0.7673**	**0.7556**	**0.4294**	**0.7780**	**0.7459**	**0.4257**

2. Compared with the homogeneous methods (GCN, GAT, HyperGCN, HHN-S), the heterogeneous network can effectively grasp richer semantics by capturing various relations in different meta-paths. The performance inferiority of homogeneous methods also demonstrates that a single meta-path is insufficient for building presentations. So heterogeneous information is necessary to consider.
3. Compared with the standard graph methods (GCN, GAT, HeteroGCN, HAN), hypergraph emphasizes the coherence of the related nodes and models this set-like structure, rather than pairwise association, which shows the great capacity in extracting high-order relations among questions.

Analysis of the intra- and inter-graph attentions. In order to investigate the effects of the proposed attention mechanism, we construct several comparison approaches to transform features upon the heterogeneous hypergraph structure in different manners.

- $\text{HHN}_{\text{intra}}$: only the intra-graph attention is used and the same importance is assigned to each subgraph.
- $\text{HHN}_{\text{inter}}$: only the inter-graph attention is used. Because there are two stage transformations in the intra-graph attention, we further consider three cases for comparison: the stage N→E means only removing the attention in the process of aggregation from nodes to hyperedges and assigning the same importance to each node. E→N means only removing the attention in that from hyperedges to nodes and assigning the same importance to each hyperedge. N↔E means removing the attention in both of them and assigning the same importance to all nodes and hyperedges.

From Table 4, we can see that the intra- and inter-graph attentions improve performance consistently on both datasets, which indicates the effectiveness of taking the attention mechanism to highlight important connective structures. Specifically, by the intra-graph attention, the proposed two-stage attention learns the impact of various nodes and hyperedges adaptively and focuses on important node-hyperedge pairs. Besides, the inter-graph attention learns the importance of relations in different subgraphs and preserves meaningful semantics for nodes. Note that, when we remove the inter-graph attention, the performance degradation is not significant compared to that caused by removing the attention in other stages. A possible reason is that these semantic-specific representations from different subgraphs are equally important so that applying the attention mechanism to them may not yield significant improvement.

Table 3
Ablation study of HHN.

Methods	ASSISTments2009	ASSISTments2017
GCN	0.8151	0.7593
GAT	0.8193	0.7606
HyperGCN	0.8230	0.7639
HeteroGCN	0.8247	0.7644
HAN	0.8275	0.7677
HHN-S	0.8258	0.7653
HHN	**0.8309**	**0.7722**

Analysis of the projection function. To validate the superiority of the proposed projection function $f_\epsilon(\cdot)$ in measuring the importance of nodes for the connected hyperedge, we substitute it with several different methods and show the results in Table 5.

- mean: assigns the same importance to each node, i.e., removing the attention in the stage N→E as described below,

$$f_\epsilon\left(\mathbf{h}_{i,\mathscr{G}_k}^{l-1}\right) = \frac{1}{|\mathscr{N}_{\epsilon,\mathscr{G}_k}|}. \tag{30}$$

- p-norm: adopts the p-norm to quantify the importance of each node. In this ablation study, we select the Euclid norm (p = 2) to conduct experiments,

$$f_\epsilon\left(\mathbf{h}_{i,\mathscr{G}_k}^{l-1}\right) = \frac{\|\mathbf{h}_{i,\mathscr{G}_k}^{l-1}\|_p}{\sum_{j\in\mathscr{N}_{\epsilon,\mathscr{G}_k}} \|\mathbf{h}_{j,\mathscr{G}_k}^{l-1}\|_p}. \tag{31}$$

As shown by Table 5, the proposed projection function outperforms other methods, which illustrates that our function is more suitable for learning the importance of a node by extracting more latent semantic information from its neighbors. The intuitive way to just consider its own features may be insufficient to represent the importance with multi-faceted impact.

Analysis of the prediction network. As mentioned before, the proposed HHN can be adapted to various downstream prediction models. To demonstrate its suitability, we select several typical time-series methods as variants of the prediction network and report their performance in Table 6.

- HHN + LSTM (DKT): supplements DKT with the proposed HHN. The refined embeddings obtained from the HHN are then fed into DKT [7] to generate final prediction.
- HHN + S-GRU: sums the embeddings of questions and responses before RNN and then applies a simple GRU to predict a student's response.
- HHN + Attention (SAKT): adopts the self-attention mechanism proposed in SAKT [11] as the prediction network.

According to the results in Table 6, we can find that HHN is capable of enhancing the downstream task performance, particularly improving DKT about 2.86% and 2.66%, SAKT about 2.21% and 3.15% on those two datasets. Such additional performance gains also indicate the effectiveness of HHN from another perspective and confirm that an adequate bottom representation can improve the downstream prediction accuracy to some extent.

Meanwhile, we can observe that the traditional self-attention method yields the worst performance in all cases. A possible reason is that the memory decay has a significant impact on students' knowledge state. In KT, due to the long learning cycle, the knowledge, which was well mastered before, may be forgotten now. Attending relative interactions simply over long sequence is adverse to prediction.

Analysis of the self-supervised framework. Here we mainly analyze the adaptive augmentation and the selection of negative samples, and evaluate their positive contributions to self-supervised learning.

- $\#U$ substitutes the adaptive schemes with the uniform counterparts.
- $\#E$ selects the views of any different nodes as the negative pairs, i.e., $\{(i,j)|i,j \in \mathscr{V}_h, i \neq j\}$.

From Table 7, we can see that our proposed method outperforms other counterparts, which confirms the superiority of the adaptive schemes. In addition, the selection of negative samples further boosts the performance. This design helps the model to capture the subtle discrepancies between those similar representations and learn more discriminative embeddings for downstream tasks.

Analysis of hyper-parameters. In order to explore the stability of S^2-HHN, we explore the influence of some critical hyper-parameters and report the results on ASSISTments2009 with various parameters in Fig. 3 and 4.

Table 4
Ablation study of intra- and inter-graph attention.

Methods	Stage	ASSISTments2009	ASSISTments2017
HHN_{intra}	G→N	0.8293	0.7710
HHN_{inter}	N→E	0.8257	0.7662
	E→N	0.8271	0.7683
	N↔E	0.8249	0.7650
Ours	–	**0.8309**	**0.7722**

Table 5
Ablation study of the projection function.

Methods	ASSISTments2009	ASSISTments2017
mean	0.8257	0.7662
p-norm	0.8243	0.7657
Ours	**0.8309**	**0.7722**

Table 6
Ablation study of prediction network.

Methods	ASSISTments2009	ASSISTments2017
HHN + LSTM (DKT)	0.8241	0.7672
HHN + S-GRU	0.8264	0.7691
HHN + P-GRU (ours)	**0.8309**	**0.7722**
HHN + Attention (SAKT)	0.8107	0.7230

- Dimension of initial question embeddings, D: The results are shown in Fig. 3a, where we can see that the model achieves the best performance when $D = 256$. The trend of raising first and then dropping indicates that a lower dimension may be inadequate to encode the semantic details while a higher dimension may cause additional redundancy for representation.
- Number of propagation layers of HHN, L: The results are shown in Fig. 3b. The model performs best when the number of layers is set to 4. When L is less than 6, the fluctuation of performance is not significant. After that, the performance starts to drop due to over-smoothing.
- Magnitude of self-supervised task, λ: The results are shown in Fig. 3c. Note that the self-supervised task is removed when λ is set to 0. We can find that the self-supervised task improves the performance of the prediction task consistently when λ is less than 2; a larger λ would result in performance degradation. The reason lies in that a large weight would enforce the model to focus on the auxiliary task excessively and the benefits brought by the self-supervised framework could be easily neutralized.
- Probability of dropout, ρ_τ, ρ_λ: The results are shown in Fig. 4. When the probability is not too large, the performance of S^2-HHN is relatively stable. If setting ρ_λ to a small value (0 or 0.1), we can find that the performance keeps growing with the increase of ρ_τ. While, for large ρ_λ (0.4 or 0.5), the performance declines precipitously. A high probability of dropout would destroy the connective structures inevitably, which leads to the incapacity of learning the structural information and optimizing the contrastive objective.

4.6. Visualization

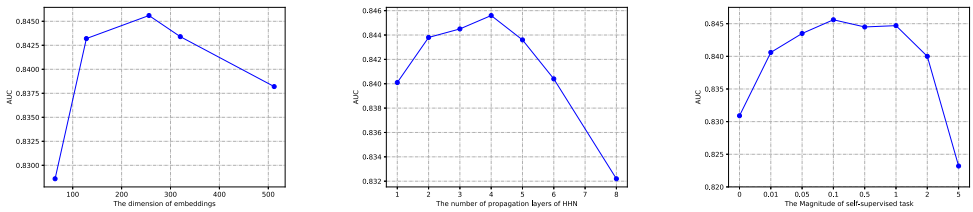
For a more intuitive comparison, we conduct the task of visualization with several state-of-the-art methods and apply t-SNE [43] to visualize the question embeddings. We randomly select five skills and the questions that cover them in ASSISTments2017. As shown in Fig. 5a, DKT does not perform well. Due to Rasch embeddings, AKT forms a curve for questions with the same skills. However, for those questions covering multiple skills, AKT tends to form isolated spots, which is adverse to building representations. In GIKT, those questions covering different skills are likely mixed with each other. From Fig. 5d, we see that our model divides nodes into five different groups, which demonstrates HHN can effectively keep intra-class similarity and alleviate the semantic blending among questions with different skills to some extent.

5. Conclusion

In this paper, we propose a novel self-supervised heterogeneous hypergraph network for knowledge tracing, named S^2-HHN. We build a heterogeneous hypergraph to capture the heterogeneous and high-order relations among questions. With the intra- and inter-graph attentions, our model can learn the importance of nodes and hyperedges. Moreover, to further enhance the representation, we supplement the network with self-supervised task and generate the augmented view via

Table 7
Ablation study of self-supervised framework.

Methods	ASSISTments2009	ASSISTments2017
S^2-HHN-EU	0.8328	0.7730
S^2-HHN-U	0.8406	0.7782
S^2-HHN-E	0.8371	0.7745
Ours	**0.8456**	**0.7803**



(a) Dimension of initial question embeddings D (b) Number of propagation layers of HHN L (c) Magnitude of self-supervised task λ

Fig. 3. AUC comparisons of HHN with different parameters.

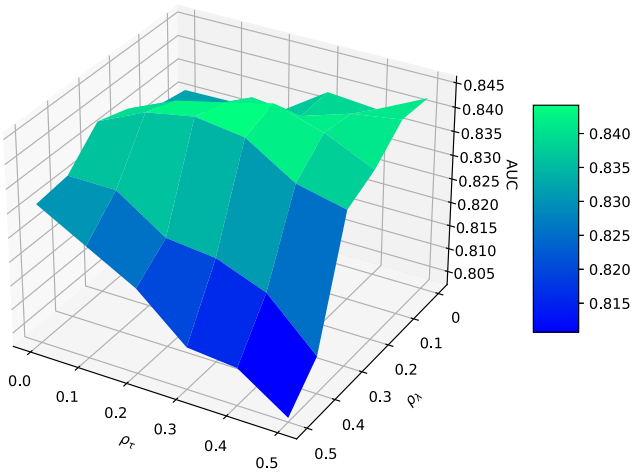


Fig. 4. The performance (AUC) of S^2-HHN with varied hyperparameters ρ_τ and ρ_λ.

adaptive paradigm to perform contrastive learning. Extensive experiments verify that our proposed model is superior to some state-of-the-art ones.

In the future, we plan to explore some other prediction networks and take the memory decay into consideration, so as to improve the performance of the long sequence prediction.

CRediT authorship contribution statement

Tangjie Wu: Conceptualization, Data curation, Formal analysis, Methodology. **Qiang Ling:** Investigation, Project administration, Funding acquisition, Supervision, Writing - review & editing.

Data availability

Public datasets were used.

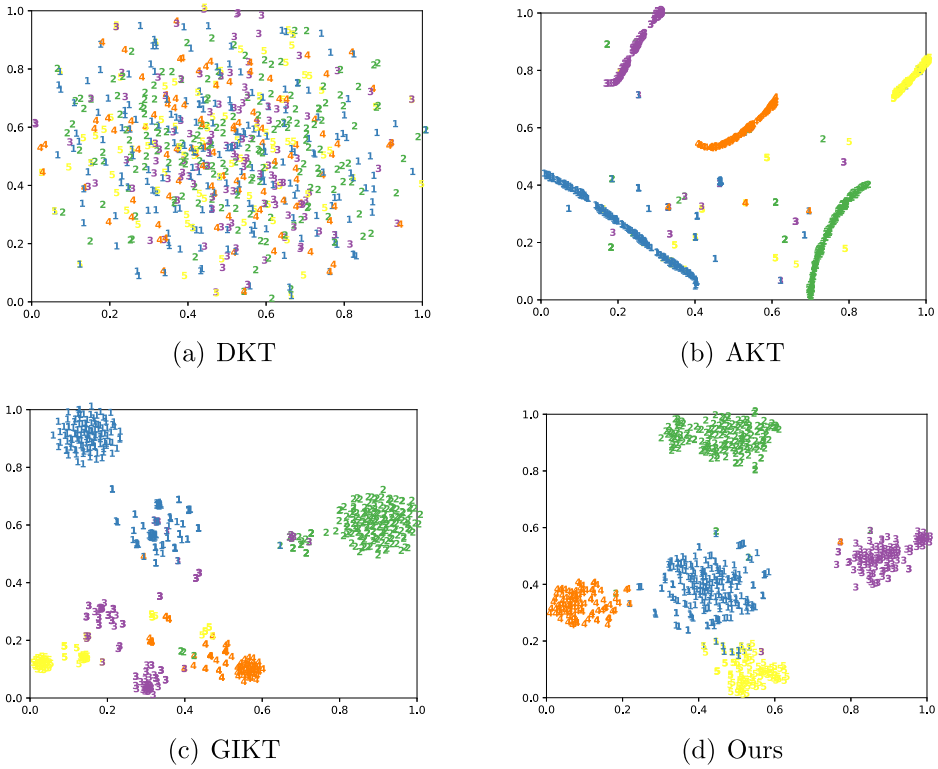


Fig. 5. Visualization embedding. Each point represents one question and its color means the skill covered in this question.

Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgements

This work was supported in part by the Key Science and Technology Program of Anhui under Grants 202203f07020002 and 202203a05020012, and in part by the Applied Science and Technology Achievement Cultivation Project of Institute of Advanced Technology, University of Science and Technology of China.

References

[1] A.C. Graesser, M.W. Conley, A. Olney, Intelligent tutoring systems., APA Educational Psychology Handbook, Vol 3: Application to Learning and Teaching. (2012) 451–473.
[2] Y. Huo, D.F. Wong, L.M. Ni, L.S. Chao, J. Zhang, Knowledge modeling via contextualized representations for LSTM-based personalized exercise recommendation, Information Sciences 523 (2020) 266–278.
[3] A.T. Corbett, J.R. Anderson, Knowledge tracing: Modeling the acquisition of procedural knowledge, User Modeling and User-Adapted Interaction 4 (4) (1994) 253–278.
[4] J. Kasurinen, U. Nikula, Estimating programming knowledge with B ayesian knowledge tracing, ACM SIGCSE Bulletin 41 (3) (2009) 313–317.
[5] W.J. van der Linden, R.K. Hambleton, Handbook of modern item response theory, Springer Science & Business Media, 2013.
[6] R.V. Lindsey, J.D. Shroyer, H. Pashler, M.C. Mozer, Improving students' long-term knowledge retention through personalized review, Psychological Science 25 (3) (2014) 639–647.
[7] C. Piech, J. Spencer, J. Huang, S. Ganguli, M. Sahami, L. Guibas, J. Sohl-Dickstein, Deep knowledge tracing, arXiv preprint arXiv:1506.05908.
[8] T. Mikolov, M. Karafiát, L. Burget, J. Černocký, S. Khudanpur, Recurrent neural network based language model, in: Eleventh Annual Conference of the International Speech Communication Association, 2010.
[9] J. Zhang, X. Shi, I. King, D.-Y. Yeung, Dynamic key-value memory networks for knowledge tracing, in: Proceedings of the 26th International Conference on World Wide Web, 765–774, 2017.
[10] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A.N. Gomez, Ł. Kaiser, I. Polosukhin, Attention is all you need, in: Advances in Neural Information Processing Systems, 5998–6008, 2017.
[11] S. Pandey, G. Karypis, A self-attentive model for knowledge tracing, arXiv preprint arXiv:1907.06837.
[12] A. Ghosh, N. Heffernan, A.S. Lan, Context-aware attentive knowledge tracing, in: Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, 2330–2339, 2020.
[13] Y. Yang, J. Shen, Y. Qu, Y. Liu, K. Wang, Y. Zhu, W. Zhang, Y. Yu, GIKT: a graph-based interaction model for knowledge tracing, in: Joint European Conference on Machine Learning and Knowledge Discovery in Databases, Springer, 299–315, 2020.

[14] X. Song, J. Li, Y. Tang, T. Zhao, Y. Chen, Z. Guan, Jkt: A joint graph convolutional network based deep knowledge tracing, Information Sciences 580 (2021) 510–523.
[15] W. Wang, H. Ma, Y. Zhao, F. Yang, L. Chang, PERM: Pre-training Question Embeddings via Relation Map for Improving Knowledge Tracing, in: International Conference on Database Systems for Advanced Applications, Springer, 2022, pp. 281–288.
[16] W. Wang, H. Ma, Y. Zhao, Z. Li, X. He, Tracking knowledge proficiency of students with calibrated Q-matrix, Expert Systems with Applications 192 (2022) 116454.
[17] Y. Sun, J. Han, Mining heterogeneous information networks: a structural analysis approach, ACM SIGKDD Explorations Newsletter 14 (2) (2013) 20–28.
[18] A. Bretto, Hypergraph theory, An introduction: Mathematical Engineering. Cham: Springer.
[19] Y. Feng, H. You, Z. Zhang, R. Ji, Y. Gao, Hypergraph neural networks, in: Proceedings of the AAAI Conference on Artificial Intelligence, vol. 33, 3558–3565, 2019.
[20] P. Velickovic, W. Fedus, W.L. Hamilton, P. Liò, Y. Bengio, R.D. Hjelm, Deep Graph Infomax., ICLR 2 (3) (2019) 4.
[21] W. Falcon, K. Cho, A framework for contrastive self-supervised learning and designing a new approach, arXiv preprint arXiv:2009.00104.
[22] Y. Zhou, Q. Liu, J. Wu, F. Wang, Z. Huang, W. Tong, H. Xiong, E. Chen, J. Ma, Modeling context-aware features for cognitive diagnosis in student learning, in: Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery & Data Mining, 2420–2428, 2021.
[23] S. Liu, J. Yu, Q. Li, R. Liang, Y. Zhang, X. Shen, J. Sun, Ability boosted knowledge tracing, Information Sciences 596 (2022) 567–587.
[24] C. Wang, W. Ma, M. Zhang, C. Lv, F. Wan, H. Lin, T. Tang, Y. Liu, S. Ma, Temporal cross-effects in knowledge tracing, in: Proceedings of the 14th ACM International Conference on Web Search and Data Mining, 517–525, 2021a.
[25] W. Wang, H. Ma, Y. Zhao, F. Yang, L. Chang, SEEP: Semantic-enhanced question embeddings pre-training for improving knowledge tracing, Information Sciences 614 (2022) 153–169.
[26] X. Wang, H. Ji, C. Shi, B. Wang, Y. Ye, P. Cui, P.S. Yu, Heterogeneous graph attention network, in: The World Wide Web Conference, 2022–2032, 2019.
[27] Y. Wang, S. Tang, Y. Lei, W. Song, S. Wang, M. Zhang, DisenHAN: Disentangled Heterogeneous Graph Attention Network for Recommendation, in: Proceedings of the 29th ACM International Conference on Information & Knowledge Management, 1605–1614, 2020a.
[28] C. Zhang, D. Song, C. Huang, A. Swami, N.V. Chawla, Heterogeneous graph neural network, in: Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, 793–803, 2019.
[29] J. Jiang, Y. Wei, Y. Feng, J. Cao, Y. Gao, Dynamic Hypergraph Neural Networks., in: IJCAI, 2635–2641, 2019.
[30] S. Ji, Y. Feng, R. Ji, X. Zhao, W. Tang, Y. Gao, Dual channel hypergraph collaborative filtering, in: Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, 2020–2029, 2020.
[31] J. Wang, K. Ding, L. Hong, H. Liu, J. Caverlee, Next-item recommendation with sequential hypergraphs, in: Proceedings of the 43rd international ACM SIGIR Conference on Research and Development in Information Retrieval, 1101–1110, 2020b.
[32] X. Xia, H. Yin, J. Yu, Q. Wang, L. Cui, X. Zhang, Self-supervised hypergraph convolutional networks for session-based recommendation, in: Proceedings of the AAAI Conference on Artificial Intelligence, vol. 35, 4503–4511, 2021.
[33] Z. Peng, W. Huang, M. Luo, Q. Zheng, Y. Rong, T. Xu, J. Huang, Graph representation learning via graphical mutual information maximization, in: Proceedings of the Web Conference 2020, 259–270, 2020.
[34] K. Hassani, A.H. Khasahmadi, Contrastive multi-view representation learning on graphs, in: International Conference on Machine Learning, PMLR, 4116–4126, 2020.
[35] Y. Zhu, Y. Xu, F. Yu, Q. Liu, S. Wu, L. Wang, Graph contrastive learning with adaptive augmentation, in: Proceedings of the Web Conference 2021, 2069–2080, 2021.
[36] X. Wang, N. Liu, H. Han, C. Shi, Self-supervised heterogeneous graph neural network with co-contrastive learning, in: Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery & Data Mining, 1726–1736, 2021b.
[37] W. Lee, J. Chun, Y. Lee, K. Park, S. Park, Contrastive Learning for Knowledge Tracing, in: Proceedings of the ACM Web Conference 2022, 2330–2338, 2022.
[38] J. Chung, C. Gulcehre, K. Cho, Y. Bengio, Empirical evaluation of gated recurrent neural networks on sequence modeling, arXiv preprint arXiv:1412.3555.
[39] Y. Ren, B. Liu, C. Huang, P. Dai, L. Bo, J. Zhang, Heterogeneous deep graph infomax, arXiv preprint arXiv:1911.08538.
[40] K. Nagatani, Q. Zhang, M. Sato, Y.-Y. Chen, F. Chen, T. Ohkuma, Augmenting knowledge tracing by considering forgetting behavior, in: The world Wide Web Conference, 3101–3107, 2019.
[41] T.N. Kipf, M. Welling, Semi-supervised classification with graph convolutional networks, arXiv preprint arXiv:1609.02907.
[42] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Lio, Y. Bengio, Graph attention networks, arXiv preprint arXiv:1710.10903.
[43] L. Van der Maaten, G. Hinton, Visualizing data using t-SNE, Journal of Machine Learning Research 9 (11).